UVM-Based Verification of I2C Protocol

Course: ECS324 VLSI Testing & Verification

Team Members:
Mukund Rathi [2023BEC0051]
Aniketh Yuvaraj C. [2023BEC0049]

Faculty Mentor:
Dr. Lakshmi N S

March 2026

Contents

1 Objective and Overview **3**
1.1 Objective . . . 3
1.2 Overview . . . 3

2 I2C Protocol Background **3**
2.1 Introduction . . . 3
2.2 Key Features . . . 3
2.3 Transaction Flow . . . 3

3 Design Description **4**
3.1 I2C Design Block Diagram . . . 4
3.2 Design Under Test (DUT) . . . 4
3.3 Interface . . . 4
3.4 RTL Synthesis and Hardware Mapping . . . 5

4 UVM Methodology **5**
4.1 Introduction . . . 5
4.2 Key Concepts . . . 6
4.3 UVM Components . . . 6
4.4 UVM Phases . . . 6

5 Verification Architecture **7**
5.1 Architecture Overview . . . 7
5.2 Component Description . . . 7
5.3 Data Flow and Operation . . . 8

6 Transaction-Level Modeling **8**

7 Test Implementation **9**
7.1 Test Case . . . 9
7.2 Expected Behavior . . . 9

8 Simulation Setup **9**

9 Simulation and Waveform Analysis **9**
9.1 Simulation Overview . . . 9
9.2 Waveform Observations . . . 10
9.2.1 Idle Condition . . . 10
9.2.2 Start Condition . . . 10
9.2.3 Address Transmission . . . 10
9.2.4 Acknowledge . . . 10
9.2.5 Data Transfer . . . 10
9.2.6 Stop Condition . . . 10
9.3 Key Observations . . . 10
9.4 Deviation . . . 10
9.5 Importance . . . 10

10 Simulation Results **11**
10.1 Analysis of UVM Output 11
10.2 Scoreboard Result 12
10.3 Interpretation 12

11 Debugging and Issues **12**

12 Key Learnings **12**

13 Future Work **12**

14 Conclusion **12**

1 Objective and Overview

1.1 Objective

The objective of this project is to design and verify an I2C communication protocol using the Universal Verification Methodology (UVM). The focus is on building a reusable and modular verification environment capable of validating protocol correctness at both transaction and signal levels. Additionally, RTL synthesis is performed to demonstrate hardware feasibility of the design and validate its suitability for FPGA implementation.

1.2 Overview

The project includes:

- Behavioral I2C design (DUT)
- Complete UVM testbench
- Gate level Synthesis
- Transaction-level verification
- Scoreboard-based validation
- Waveform-based protocol analysis

2 I2C Protocol Background

2.1 Introduction

I2C (Inter-Integrated Circuit) is a synchronous serial communication protocol used for communication between integrated circuits using two wires: SDA (data) and SCL (clock).

2.2 Key Features

- Two-wire interface
- Open-drain signaling
- Master-slave architecture
- Acknowledgment-based transfer

2.3 Transaction Flow

1. START condition
2. Address + R/W bit
3. ACK from slave
4. Data transfer
5. STOP condition

3 Design Description

3.1 I2C Design Block Diagram

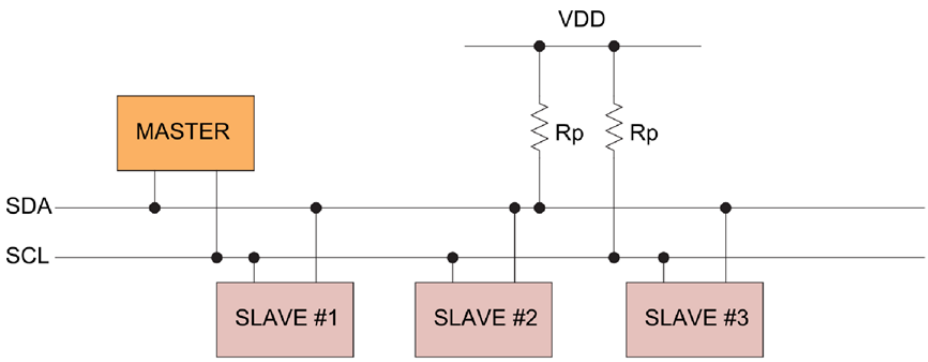

Figure 1: I2C Bus Architecture Showing Master, Multiple Slaves, and Pull-Up Resistors

- The I2C (Inter-Integrated Circuit) protocol is a two-wire serial communication protocol that uses a shared bus consisting of Serial Data Line (SDA) and Serial Clock Line (SCL). A single master device controls the communication by generating the clock signal, while multiple slave devices are connected to the same bus.

- Both SDA and SCL lines are open-drain and require pull-up resistors to maintain a default high state. Communication is initiated by the master using a START condition, followed by a 7-bit address and a read/write bit. The addressed slave responds with an acknowledgment (ACK), after which data transfer occurs.

- This architecture allows multiple slave devices to communicate efficiently over the same bus with minimal wiring.

3.2 Design Under Test (DUT)

The DUT implements a simplified I2C master controller with an FSM-based architecture responsible for generating START/STOP conditions, transmitting address and data, and handling protocol sequencing. It processes address and data fields and generates acknowledgments.

3.3 Interface

The interface `i2c_if` abstracts SDA and SCL lines, enabling clean interaction between DUT and UVM components.

3.4 RTL Synthesis and Hardware Mapping

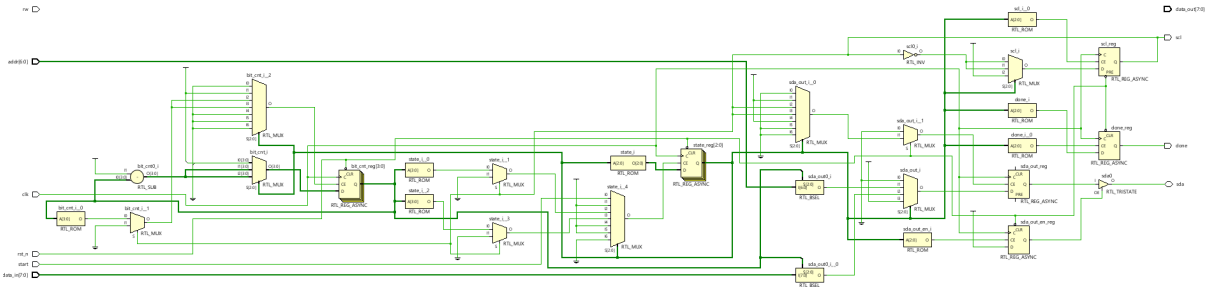

Figure 2: RTL Schematic of the Synthesized I2C Master Design in Vivado

- The synthesized RTL schematic of the I2C master design is shown in Figure above. The schematic represents the hardware-level implementation generated by the Vivado synthesis tool from the RTL description.

- The design is primarily composed of sequential elements such as registers and combinational logic blocks including multiplexers and control logic. The state machine responsible for controlling the I2C protocol is implemented using state registers and next-state logic, which can be observed in the schematic as state_reg and associated multiplexers.

- A bit counter is used to track the number of bits transmitted during address and data phases, ensuring correct serial communication. The presence of multiplexers indicates data selection between different phases such as address transmission and data transfer.

- The SDA line is implemented using tri-state logic, which reflects the open-drain behavior of the I2C protocol. This allows both master and slave devices to share the bus without contention.

The synthesis results confirm that the design is fully hardware realizable and suitable for FPGA implementation. The mapping of RTL constructs into hardware primitives demonstrates correct structural realization of the I2C controller.

4 UVM Methodology

4.1 Introduction

UVM is an industry-standard verification framework based on SystemVerilog that enables modular, scalable, and reusable testbench design.

4.2 Key Concepts

- Transaction-Level Modeling (TLM)
- Component hierarchy
- Separation of stimulus and checking

4.3 UVM Components

Sequence Item: Defines transaction (address, data, direction)
Sequencer: Controls flow of transactions
Driver: Converts transactions to signal-level activity
Monitor: Observes signals and reconstructs transactions
Scoreboard: Validates correctness
Agent: Combines driver, monitor, sequencer
Environment: Top-level integration
Test: Controls execution using `run_test()`

- The UVM environment ensures separation between stimulus generation and checking, enabling scalable and reusable verification. This modular approach allows independent development and debugging of components.

4.4 UVM Phases

UVM follows a phase-based execution mechanism that ensures structured simulation flow:

- **Build Phase**: Components are created and configured.
- **Connect Phase**: TLM connections between components are established.
- **Run Phase**: Stimulus is generated and applied to the DUT.
- **Extract Phase**: Data is collected after simulation.
- **Report Phase**: Final results are summarized.

5 Verification Architecture

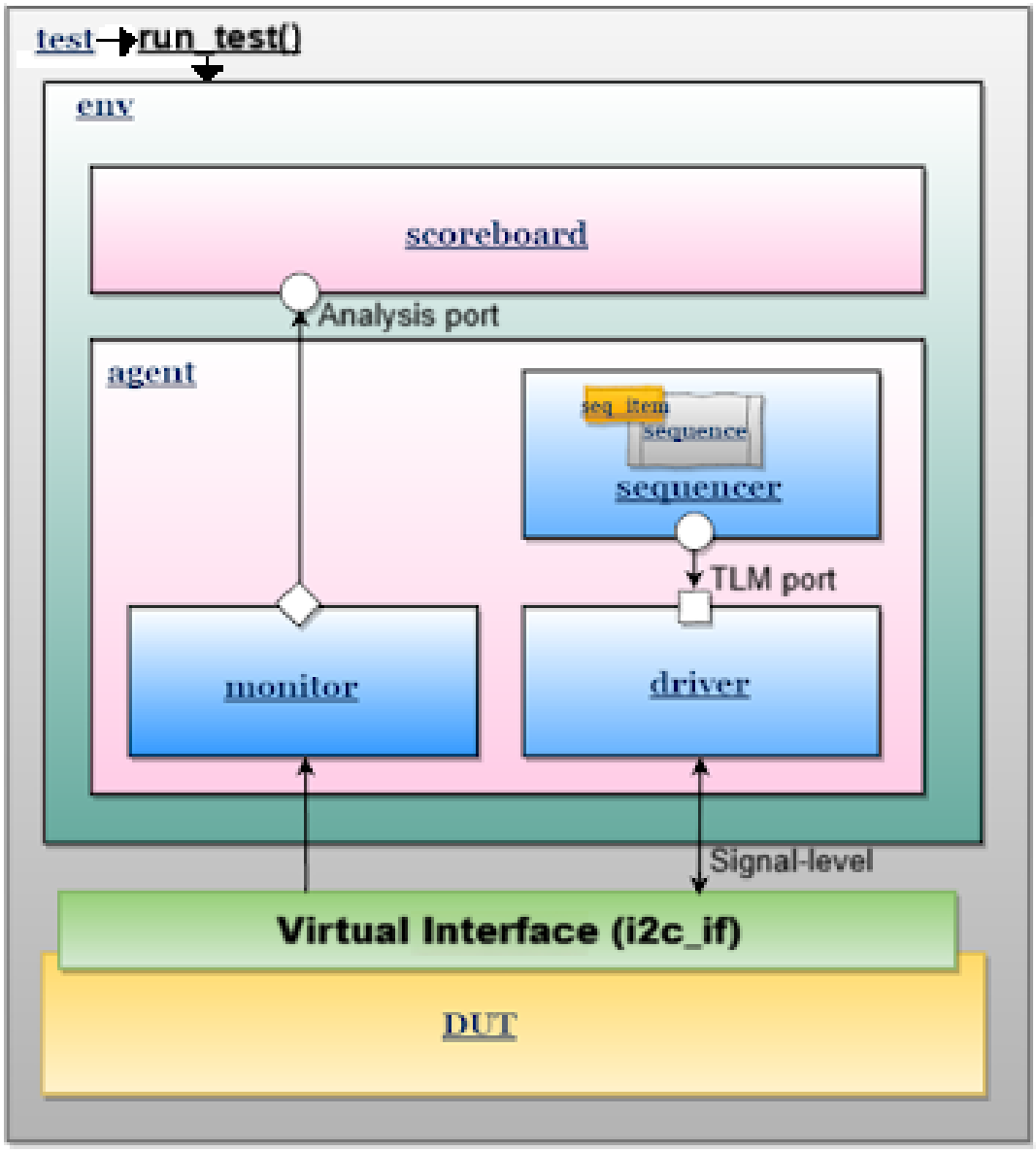

Figure 3: UVM Testbench Architecture for I2C Controller Verification

5.1 Architecture Overview

The UVM testbench follows a hierarchical structure consisting of the test, environment, agent, and supporting verification components. The `run_test()` mechanism is used to initiate the verification flow.

5.2 Component Description

- **Test**: Top-level control component that configures the environment and starts stimulus generation using `run_test()`.

- **Environment (env)**: Contains all verification components including agent and scoreboard.
- **Agent**: Encapsulates the **sequencer**, **driver**, and **monitor**.
- **Sequencer**: Generates high-level transactions (sequence items) representing I2C operations.
- **Driver**: Converts transactions into signal-level activity and drives the DUT through the virtual interface (`i2c_if`).
- **Monitor**: Passively observes signal activity from the interface and reconstructs transactions without affecting the DUT.
- **Scoreboard**: Receives transactions from the monitor via analysis ports and checks correctness of DUT behavior.
- **Virtual Interface**: Acts as a bridge between the testbench and DUT, enabling signal-level communication abstraction.
- **DUT**: The I2C controller being verified.

5.3 Data Flow and Operation

The UVM testbench architecture follows a structured data flow:

- The test invokes `run_test()` to start the simulation.
- The sequencer generates sequence items which are sent to the driver using Transaction-Level Modeling (TLM).
- The driver translates these transactions into signal-level operations and drives the DUT via the virtual interface.
- The monitor passively observes the interface signals and reconstructs the transactions.
- The reconstructed transactions are forwarded to the scoreboard through analysis ports.
- The scoreboard compares expected and observed results to verify functional correctness.

6 Transaction-Level Modeling

UVM uses TLM for communication:

- Driver uses seq_item_port
- Monitor uses analysis port
- Scoreboard uses analysis export

This ensures modular and reusable verification.

7 Test Implementation

7.1 Test Case

- Address: 0x50
- Operation: WRITE
- Data: 0xA5, 0x5A

Parameter	Value
Address	0x50
Operation	WRITE
Data Bytes	0xA5, 0x5A
Result	PASS

Table 1: Test Case Summary

The above table summarizes the key parameters of the verification scenario. The successful PASS result indicates that the DUT correctly executed the I2C WRITE transaction.

7.2 Expected Behavior

- START condition generated
- Address decoded
- Data transmitted correctly
- STOP condition generated

8 Simulation Setup

```
qrun -batch -uvmhome uvm-1.2 design.sv testbench.sv -do run.do
```

Simulation is executed using QuestaSim with UVM 1.2 support.

9 Simulation and Waveform Analysis

9.1 Simulation Overview

The simulation executed a WRITE transaction with address 0x50 and two data bytes. The execution completed successfully without errors.

9.2 Waveform Observations

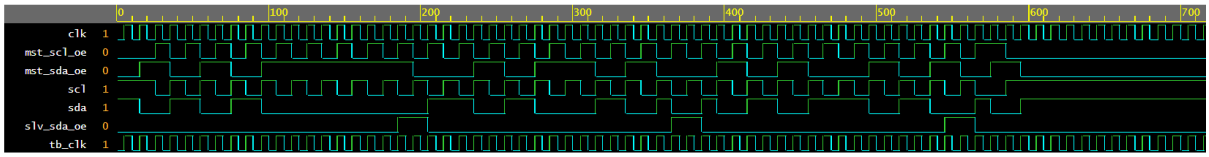

Figure 4: I2C Waveform Showing START, Address, Data, ACK, and STOP Conditions

9.2.1 Idle Condition

Both SDA and SCL remain HIGH, representing idle state.

9.2.2 Start Condition

SDA transitions from HIGH to LOW while SCL is HIGH, indicating START.

9.2.3 Address Transmission

Address bits are transmitted on SDA, synchronized with SCL.

9.2.4 Acknowledge

Slave pulls SDA LOW to acknowledge.

9.2.5 Data Transfer

Data bytes 0xA5 and 0x5A are transmitted bit-by-bit.

9.2.6 Stop Condition

SDA transitions from LOW to HIGH while SCL is HIGH.

9.3 Key Observations

- SDA changes only when SCL is LOW
- Data stable when SCL is HIGH
- ACK correctly generated

9.4 Deviation

SCL toggles continuously, which is a simplification acceptable for functional verification.

9.5 Importance

Waveform analysis validates signal-level correctness complementing UVM transaction-level checks.

- The observed waveform behavior strictly adheres to the I2C protocol specification, confirming correct temporal and logical sequencing of signals.

10 Simulation Results

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   13
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [DRV]     1
# [MON]     1
# [Questa UVM]     2
# [RNTST]     1
# [SCB]     2
# [SLV]     4
# [TEST_DONE]     1
# [UVM/RELNOTES]     1
#
# ** Note: $finish    : /usr/share/questa/questasim/verilog_src/uvm-1.2/src/base/uvm_root.svh(517)
#    Time: 805 ns  Iteration: 69  Instance: /tb_top
# End time: 12:50:07 on Mar 28,2026, Elapsed time: 0:00:03
# Errors: 0, Warnings: 0
End time: 12:50:07 on Mar 28,2026, Elapsed time: 0:00:17
*** Summary ********************************************
    qrun: Errors:   0, Warnings:   0
    vlog: Errors:   0, Warnings:   0
    vopt: Errors:   0, Warnings:   0
    vsim: Errors:   0, Warnings:   0
  Totals: Errors:   0, Warnings:   0
Finding VCD file...
./dump.vcd
[2026-03-28 16:50:08 UTC] Opening EPWave...
Done
```

Figure 5: UVM Simulation Log Showing Transaction Execution and Scoreboard Result

The UVM simulation log, absence of UVM_ERROR and UVM_FATAL messages confirms correct execution of the verification environment without functional mismatches. The driver initiates the transaction, the slave responds appropriately, and the monitor reconstructs the transaction. The scoreboard validates correctness and reports a PASS with zero errors or warnings.

10.1 Analysis of UVM Output

The simulation log provides a detailed view of the transaction-level behavior of the verification environment.

- The driver initiates the transaction with address 0x50 and two data bytes.
- The slave detects the START condition and processes the WRITE operation.
- Data bytes 0xA5 and 0x5A are successfully received.
- The monitor reconstructs the transaction from signal activity.
- The scoreboard compares expected and observed data and reports a PASS.

10.2 Scoreboard Result

- PASS: 1
- FAIL: 0

10.3 Interpretation

The absence of errors or warnings confirms that the verification environment is functioning correctly. The successful scoreboard validation indicates that the DUT behavior matches the expected protocol operation.

This demonstrates correctness at the transaction level, complementing the signal-level validation performed through waveform analysis.

- PASS: 1
- FAIL: 0

11 Debugging and Issues

- Package compilation errors resolved
- Missing fields corrected
- Test execution fixed using explicit test name

12 Key Learnings

- UVM enables modular design
- TLM simplifies communication
- Scoreboards ensure correctness

13 Future Work

- Functional coverage can be added to measure verification completeness.
- Add READ transactions
- Add coverage metrics
- Multi-master verification

14 Conclusion

This project successfully demonstrates UVM-based verification of the I2C protocol. Both transaction-level and signal-level analysis confirm correctness of the implementation. The integration of RTL design, UVM-based verification, and synthesis demonstrates a complete VLSI design and verification flow.